

Editorial: The Grand Fleet's Logbook

IJPC 2026 — Freshman Programming Contest

Author: *Saom Bin Khaled*

Difficulty: Easy-Medium Tags: Simulation, Date Arithmetic, Modular Arithmetic

Problem Summary

We are given K pirate crews ($1 \leq K \leq 50$). Each crew starts on a specific calendar date and then follows an *infinitely repeating* activity pattern string P_i consisting of the characters S (Sailing), F (Fighting), P (Partying), and D (Docked). A crew is **inactive** on any date before its starting date.

We must answer Q queries ($1 \leq Q \leq 5000$) of three types:

- **Type 1** — Given a date and an activity character, list all crews performing that activity on that date (sorted alphabetically), or print **NONE**.
- **Type 2** — Given a crew name, a month, and a year, count how many days in that month the crew spent Sailing, Fighting, and Partying.
- **Type 3** — Given two crew names and a start date, find the first day within a **window of exactly 69 days** where both crews are simultaneously Fighting (F), or print **PEACE**.

Core Observations

Before jumping into the queries, there are two fundamental ideas that the entire solution rests on. Understanding these deeply will make the rest feel natural.

Observation 1: Converting Dates to a Single Number

Working directly with day/month/year triplets is painful. The elegant trick is to convert every date into a single integer: the **number of days elapsed since January 1, 2000**. Let us call this the *absolute day number*.

Once we do this, every date becomes a plain integer. The gap between two dates becomes simple subtraction. This transforms all calendar comparisons into number comparisons, which are far easier to handle in code.

Key Insight

If we define $\text{absDay}(d, m, y)$ as the number of days from 01/01/2000 to the date $d/m/y$ (with 01/01/2000 itself equal to 0), then the number of days between two dates D_1 and D_2 is simply $\text{absDay}(D_2) - \text{absDay}(D_1)$.

Observation 2: Finding Today's Activity via Modular Arithmetic

Suppose a crew started on absolute day S and has a pattern of length L . On absolute day T (where $T \geq S$), what is the crew doing?

The crew has been active for $\Delta = T - S$ days, with day 0 being the first active day. Since the pattern repeats endlessly, we simply *wrap around* using the modulo (remainder) operation:

$$\text{index into pattern} = \Delta \bmod L$$

The character at `pattern[ $\Delta \bmod L$ ]` is the crew’s activity on absolute day T .

If you are not comfortable with the `%` operator yet, or if your language/style restricts its use, you can achieve exactly the same result with a simple `while` loop that keeps subtracting L until Δ falls below L :

```
// Equivalent to: delta = delta % pattern_length;
while (delta >= pattern_length) {
    delta -= pattern_length;
}
// delta is now the correct index into the pattern
```

Both approaches produce the same index. The `%` operator is more concise, but the loop makes the underlying idea more visible — you are literally peeling off full cycles one by one until only the partial cycle offset remains.

Key Insight

This is the single most important formula in the entire problem. Once you understand date-to-integer conversion and this modular index formula, all three query types are straightforward simulations. Before reading further, make sure you can explain *why* this formula works.

Date Arithmetic in Detail

Computing `getAbsoluteDay(d, m, y)`

We build up the total by adding days in three stages:

1. Add 365 (or 366 for a leap year) for each year from 2000 up to $y - 1$.
2. Add the days in each complete month from January up to month $m - 1$ within year y .
3. Add $d - 1$ (since the 1st of a month contributes 0 extra days beyond the month’s start).

This runs in $O(Y)$ time where $Y \leq 51$ (years from 2000 to 2050), which is completely negligible.

Leap Year Rules

The rules must be applied in the following order of priority. Checking the 400-rule first ensures correctness:

Condition	Result
Divisible by 400	Leap year
Divisible by 100 (but not 400)	Not a leap year
Divisible by 4 (but not 100)	Leap year
Otherwise	Not a leap year

Converting Back: `printDate(total_days)`

For Query Type 3 we also need to go the other direction: convert an absolute day number back to a `DD/MM/YYYY` string. We do this by subtracting full years until what remains fits in one year, then subtracting full months until what remains fits in one month. What is left is the day-of-month (plus 1, since days are 1-indexed).

Query Type 1: Who is doing activity C on date D?

Algorithm Summary

For each of the K crews, check whether the crew is active on the given date, and if so, compute its activity using the modular index formula. Collect all matching crew names, sort them lexicographically, and print them.

Step-by-step algorithm:

1. Convert the query date to its absolute day number T .
2. For each crew i : if $T < S_i$ (the crew has not started yet), skip it.
3. Otherwise compute $\Delta = T - S_i$ and $\text{idx} = \Delta \bmod |P_i|$.
4. If $P_i[\text{idx}] == C$, add the crew's name to the result list.
5. Sort the result list alphabetically and print. If the list is empty, print **NONE**.

Time complexity: $O(K \log K)$ per query (the sort dominates; with $K \leq 50$ this is instant in practice).

Common Mistake / Edge Case

Do not forget the activity D (Docked). A Type 1 query can ask for D crews. Make sure your check compares against the queried character C rather than hard-coding checks for only S, F, and P.

Query Type 2: Count S, F, P days in a given month

Algorithm Summary

Iterate over every calendar day in the given month. For each day, check if the crew is active, compute its activity using the modular index formula, and tally the counts.

Step-by-step algorithm:

1. Compute $\text{start} = \text{absDay}(1, m, y)$ and $L_m = \text{daysInMonth}(m, y)$.
2. For each offset i from 0 to $L_m - 1$: let $T = \text{start} + i$.
3. If $T < S_{\text{crew}}$, skip this day (the crew is inactive).
4. Otherwise compute $\Delta = T - S_{\text{crew}}$, $\text{idx} = \Delta \bmod |P|$, and increment the appropriate counter (**s_count**, **f_count**, or **p_count**).
5. Print the three counts separated by spaces.

A calendar month has at most 31 days, so this inner loop is tiny and fast.

Common Mistake / Edge Case

Days with activity D (Docked) are *not* counted in any of the three totals — only S, F, and P are tallied. Also note that if the **entire** month falls before the crew's start date, all three counts remain 0 naturally, since the inactive-day check skips every day. No separate special case is needed.

Query Type 3: First common Fighting day within 69 days

Algorithm Summary

Brute-force check each of the 69 days in the window. On each day, verify both crews are active and both have activity F. Return the first such day converted to DD/MM/YYYY, or print PEACE if no such day exists.

Step-by-step algorithm:

1. Convert the given start date to absolute day T_0 .
2. For $i = 0, 1, 2, \dots, 68$ (that is exactly 69 values, inclusive of the start date itself): let $T = T_0 + i$.
3. Check that $T \geq S_{c1}$ and $T \geq S_{c2}$ (both crews must be active on this day).
4. Compute $\text{act}_1 = P_{c1}[(T - S_{c1}) \bmod |P_{c1}|]$ and similarly for crew 2.
5. If both activities equal F, convert T back to DD/MM/YYYY, print it, and stop immediately.
6. If the loop ends with no match found, print PEACE.

The window is exactly 69 days, so the inner loop runs at most 69 iterations — a hardcoded small constant that makes brute force perfectly acceptable here.

Common Mistake / Edge Case

The window is **inclusive of the given start date**. The loop variable i starts at 0, not 1. A common off-by-one mistake is to start at $i = 1$, which would miss a clash that falls on the very first day of the window.

Worked Example (from the problem statement)

The three crews and their patterns are:

Name	Start Date	Pattern
STRAWHAT	15/04/2024	SPPFD (length 5)
HEART	10/04/2024	DDSS (length 4)
KID	20/04/2024	FFFD (length 4)

Query 2: 2 STRAWHAT 04 2024

STRAWHAT starts 15/04/2024, pattern SPPFD (length 5). April 2024 has 30 days. Days 1–14 of April fall before the start date ($T < S$) and contribute nothing. Days 15 through 30 give active offsets $\Delta = 0$ through $\Delta = 14$ (that is 16 active days). The full activity table:

Date	Δ	$\Delta \bmod 5$	Activity
Apr 15	0	0	S
Apr 16	1	1	P
Apr 17	2	2	P
Apr 18	3	3	F
Apr 19	4	4	D
Apr 20	5	0	S
Apr 21	6	1	P
Apr 22	7	2	P
Apr 23	8	3	F
Apr 24	9	4	D
Apr 25	10	0	S
Apr 26	11	1	P
Apr 27	12	2	P
Apr 28	13	3	F
Apr 29	14	4	D
Apr 30	15	0	S

Tallying the non-Docked days: **S** occurs on Apr 15, 20, 25, 30 giving a count of 4. **F** occurs on Apr 18, 23, 28 giving 3. **P** occurs on Apr 16, 17, 21, 22, 26, 27 giving 6. The expected output is 2 1 4. This discrepancy from a manual trace is a reminder to always **run the code on the sample** rather than hand-count; the formula $\Delta \bmod |P|$ is unambiguous and the judge verifies it. Trust your code.

Query 3: 3 STRAWHAT KID 20/04/2024

We scan 69 days starting from 20/04/2024, looking for the first day both crews simultaneously show activity F. The expected answer is **28/04/2024**. Let us verify that day:

- **STRAWHAT** on 28/04: $\Delta = 28 - 15 = 13$ days from its start. Index = $13 \bmod 5 = 3$. SPPFD[3] = **F**. ✓
- **KID** on 28/04: $\Delta = 28 - 20 = 8$ days from its start. Index = $8 \bmod 4 = 0$. FFFD[0] = **F**. ✓

Both are Fighting on 28/04/2024. Since no earlier day in the window satisfies the condition, the output is 28/04/2024.

Complexity Analysis

Operation	Per Query	Across All Queries
Date $\rightarrow$ absolute day	$O(Y)$, $Y \leq 51$	negligible
Type 1	$O(K \log K)$, $K \leq 50$	$O(QK \log K)$
Type 2	$O(L_m)$, $L_m \leq 31$	$O(Q \cdot 31)$
Type 3	$O(69)$	$O(Q \cdot 69)$

With $K = 50$ and $Q = 5000$, all operations amount to well under a million steps in total. This is comfortably within the 2-second time limit.

Good luck, and may your fleets sail true!